ФСП / КАРЬЕРА

# FSP Career

## Техническая документация

Платформа оценки навыков, объяснимого подбора
и взаимодействия специалистов с работодателями

**Версия документа:** 1.0
**Актуальность:** 2026-10-11
**Ревизия исходного кода:** 0a0b91e682b5
**API:** OpenAPI 3.0.3 · версия 2.0.0
**Состав:** архитектура · PostgreSQL · авторизация · алгоритмы · API · эксплуатация

github.com/06ED/fsp-career
brokenbrain.online

Документ подготовлен по реализации и конфигурации репозитория. Демонстрационные записи ФСП являются синтетическими данными.

# Содержание

1 Назначение и границы системы ..... 4
1.1 Роли и сценарии ..... 4
1.2 Ключевые инварианты ..... 4
1.3 Стек ..... 4
2 Архитектура и структура репозитория ..... 5
2.1 Компонентная схема ..... 5
2.2 Контексты API ..... 5
2.3 Слои и направление зависимостей ..... 6
2.4 Транзакции и сериализация ..... 6
3 Frontend и состояние приложения ..... 6
3.1 Организация кода ..... 6
3.2 Маршруты кабинета ..... 6
3.3 Авторизованные запросы ..... 7
3.4 UI и PDF-экспорт ..... 7
4 Модель данных PostgreSQL ..... 7
4.1 Группы таблиц и ключи ..... 7
4.2 Логические связи ..... 8
4.3 Ограничения и конкурентный доступ ..... 8
4.4 Миграции ..... 9
5 Идентификация, сессии и доступ ..... 9
5.1 Локальная регистрация ..... 9
5.2 Подтверждение email ..... 9
5.3 Cookie и CSRF ..... 10
5.4 ФСП ID / OIDC и VK ID ..... 10
5.5 Прикладные роли и владение ..... 10
6 Профили, приватность и резюме ..... 11
6.1 Профиль и согласия ..... 11
6.2 Приватность и контакты ..... 11
6.3 Импорт PDF ..... 11
6.4 Экспорт и отзыв согласия ..... 11
7 Квалификационная оценка: IRT и CAT ..... 11
7.1 Формат и жизненный цикл ..... 11
7.2 Модель вероятности и оценка ..... 12
7.3 Подтверждение грейда ..... 12
7.4 Пересдачи ..... 13
7.5 Blueprint по вакансии ..... 13
8 Объяснимый подбор кандидатов ..... 13
8.1 Подготовка сигналов и фильтры ..... 13
8.2 Рейтинг по потребности ..... 14
8.3 Покрытие стека и согласованность ..... 14
8.4 Банк кандидатов и пагинация ..... 14
9 Приглашения, вакансии и регулярные задания ..... 15
9.1 Приглашения ..... 15
9.2 Вакансии и отклики ..... 15
9.3 Еженедельное назначение ..... 15
10 Реестр достижений и интеграционные порты ..... 16
10.1 Текущий адаптер ФСП ..... 16
10.2 SMTP ..... 16
10.3 PDF и внешняя идентичность ..... 16
11 HTTP API и примеры контрактов ..... 16
11.1 Общие правила ..... 16

11.2 Получение сессии и изменение профиля ... 17
11.3 Старт и ответ в оценке ... 17
11.4 Поиск и предложение ... 17
11.5 Основные типы ... 18
12 Конфигурация и локальный запуск ... 18
12.1 Переменные окружения ... 18
12.2 Полный локальный стек ... 19
12.3 Разработка вне контейнеров ... 19
13 Развёртывание и CI/CD ... 20
13.1 Активная топология ... 20
13.2 Pipeline GitHub Actions ... 20
13.3 Релиз и откат ... 21
14 Тестирование и техническая валидация ... 21
14.1 Уровни проверок ... 21
14.2 Команды ... 21
14.3 Регрессионные сценарии ... 22
15 Эксплуатация и диагностика ... 22
15.1 Проверки окружения ... 22
15.2 Типовые причины ... 22
15.3 Сопровождение данных ... 23
16 Развитие и поддержка документации ... 23
16.1 Точки изменения ... 23
16.2 Источники и сборка PDF ... 23
16.3 Ссылки ... 24
17 Приложение: каталог HTTP API ... 25

Документ предназначен для разработчиков, технических экспертов и сопровождающих систему. Полный машинно-читаемый контракт находится в `apps/api/src/main/resources/openapi.yaml`; каталог операций в конце PDF формируется из этого файла при сборке.

# 1 Назначение и границы системы

FSP Career объединяет профиль специалиста, квалификационную оценку и подбор под задачи команды. Основная категория — пара «специализация + подтверждённый грейд». Самооценка и текст резюме помогают описать опыт, но не присваивают категорию.

## 1.1 Роли и сценарии

| Роль | Основные возможности |
|---|---|
| Специалист `candidate` | Профиль и согласия, импорт/экспорт резюме, приватность, опрос и CAT-тест, достижения, приглашения, поиск вакансий, отклики, задания недели. |
| Работодатель `employer` | Компания и контакты, потребность команды, ранжированная подборка и банк, карточка специалиста, предложения с зарплатной вилкой, вакансии, решения по откликам, короткие задания и оценка ответов. |

Роль выбирается один раз и хранится в PostgreSQL. До выбора внешняя/локальная идентичность может существовать без прикладного аккаунта. Повтор того же выбора идемпотентен; смена роли отклоняется. Для демонстрации двух ролей используются два аккаунта.

## 1.2 Ключевые инварианты

- Обработка и публикация персональных данных — отдельные согласия.
- Подбор включает опубликованных специалистов с подтверждённой категорией в текущей специализации.
- В сценарии приглашений контакты доступны после принятия предложения конкретного работодателя.
- Самостоятельный отклик раскрывает контакты компании, на чью вакансию он отправлен.
- Скрытые разделы профиля исключаются из выдачи и расчёта факторов.
- Тестирование и задания работодателей — разные механизмы; оценка короткого задания не повышает квалификационный грейд.
- Контактный email профиля редактируется отдельно от email входа и не подтверждает владение идентичностью ФСП.

## 1.3 Стек

| Компонент | Реализация |
|---|---|
| Web | Svelte 5, SvelteKit 2, TypeScript, Vite 7, adapter-node; сборщик bun@1.4.2 native CSS, GSAP; pdfmake для экспорта резюме. |
| API | Kotlin 2.1.20, JDK 21, Ktor 3.2.3/Netty, kotlinx.serialization, Ktor DI. |
| Хранение | PostgreSQL, Exposed 0.61.0, HikariCP 6.2.1, Flyway 11.3.4. Compose использует PostgreSQL 16; сервис интеграционных тестов CI — 18. |
| Интеграции | Keycloak/OIDC как адаптер ФСП ID, VK ID OAuth, SMTP/Mailpit, PDFBox 3.0.4 для импорта PDF. |
| Доставка | Docker, GitHub Actions, GHCR, Kubernetes/K3s, Flux, Kustomize, SOPS/age, Envoy Gateway. |

Источник: `apps/web/package.json`, `apps/api/build.gradle.kts`, Dockerfile, Compose и workflow CI/CD.

# 2 Архитектура и структура репозитория

## 2.1 Компонентная схема

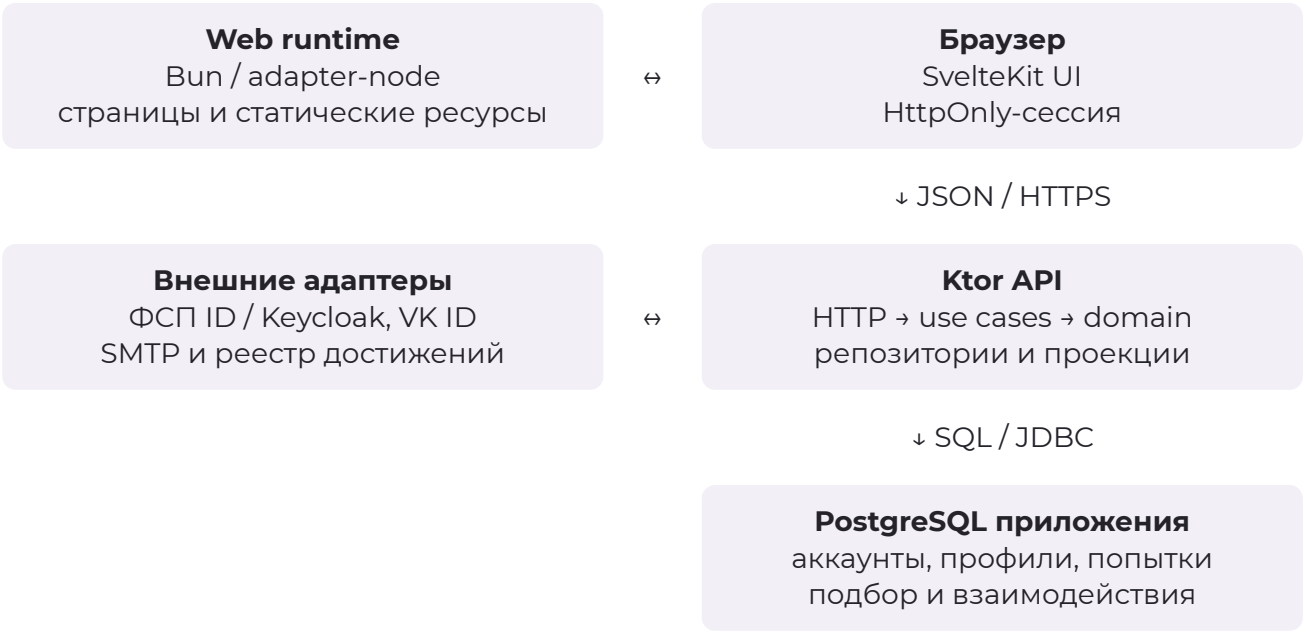



Рис. 1. Основные компоненты и границы обработки данных.

API является модульным монолитом: доменные контексты разделены пакетами, но работают в одном процессе и с общей PostgreSQL. Это позволяет атомарно выполнять проверки владения ресурсами, запись результата теста и операции отзыва согласия без распределённых транзакций.

```
apps/web/                 SvelteKit, пользовательский интерфейс
apps/api/                 Kotlin/Ktor, бизнес-логика и OpenAPI
infra/keycloak/           realm и тема провайдера ФСП ID
infra/deploy/             доставка, promotion и валидация manifests
deploy/homelab/prod/      namespace-scoped Kustomize-конфигурация
.github/workflows/        CI/CD
docs/                    документация и исходники PDF
compose.yaml             локальные web/API/DB/Keycloak/Mailpit
compose.prod.yaml        прежний Compose-вариант размещения
```

## 2.2 Контексты API

| Контекст | Ответственность |
|---|---|
| `identity` | Идентичности, локальные пароли, подтверждение email, cookie-сессии, OAuth и единственный выбор роли. |
| `profiles` | Специалист, компания, потребность, согласия, приватность, импорт и проверка резюме. |
| `assessment` | Каталоги, опрос, семейства заданий, IRT/CAT, результаты и пересдачи. |
| `matching` | Сигналы профиля, текстовая близость, категории, рейтинг, объяснения, банк и пагинация. |
| `recruitment` | Приглашения, вакансии, отклики и регулярные короткие задания. |
| `achievements` | Проверка принадлежности участника, порт реестра и импорт результатов ФСП. |

| | |
|---|---|
| `shared` | Доменные ошибки, HTTP-отображение ошибок, подключение базы, демонстрационные CLI. |

### 2.3 Слои и направление зависимостей

```
http → application → domain
infrastructure → domain / application ports
*Dependencies.kt → конкретные адаптеры и сервисы
```

`domain` содержит модели, бизнес-политики и порты. Он не зависит от Ktor, SQL, Exposed или PDFBox. `application` реализует use cases через интерфейсы; `infrastructure` реализует репозитории, DAO и интеграции; `http` принимает запрос, определяет субъекта, вызывает use case и возвращает ответ.

`Application.kt` подключает плагины и маршруты. `Dependencies.kt` объединяет модули DI, а `PlatformDependencies.kt` задаёт общие Database, AuthConfig и HttpClient. Ktor DI управляет жизненным циклом ресурсов. Ограничения слоёв проверяются `ArchitectureTest`.

### 2.4 Транзакции и сериализация

HikariCP использует максимум 8 соединений и минимум 1. На старте `Database` запускает Flyway, затем подключает Exposed. Операции выполняются в транзакциях на `Dispatchers.IO`; DAO и `EntityID` остаются внутри infrastructure, наружу возвращаются доменные значения. Списки и подбор читаются SQL-проекциями с пакетными запросами.

Ktor использует `ignoreUnknownKeys=false` и `encodeDefaults=true`. Неизвестные JSON-поля отклоняются, а значения по умолчанию присутствуют в ответах. Серверные поля ответов не следует механически отправлять как тело другого запроса.

## 3 Frontend и состояние приложения

### 3.1 Организация кода

Frontend разделён на `pages`, `widgets`, `features`, `entities` и `shared`. Маршруты SvelteKit в `src/routes` связывают страницу с feature-компонентами. В `entities` находятся типы и API-клиенты предметных сущностей; `shared` содержит авторизацию, HTTP-обёртки, UI и стили.

`createDashboard` в `pages/dashboard/model/dashboard.svelte.ts` хранит профиль, компанию, потребность, приглашения и текстовые представления списков. Контекст Svelte передаёт это состояние дочерним страницам. Локальные состояния теста, поиска, вакансий и заданий принадлежат соответствующим features.

### 3.2 Маршруты кабинета

| Маршрут | Роль | Назначение |
|---|---|---|
| `/` | Публичный | Лендинг. |
| `/login` | Публичный | Вход, регистрация, подтверждение email. |
| `/app/role` | До выбора | Единственный выбор роли. |
| `/app` | Обе | Обзор и маршрут пользователя. |
| `/app/profile` | Специалист | Профиль, резюме, приватность, интеграции. |

| | | |
|---|---|---|
| `/app/assessment` | Специалист | Опрос, квалификационная оценка, история и blueprint. |
| `/app/company` | Работодатель | Компания и способы связи. |
| `/app/needs` | Работодатель | Потребность команды. |
| `/app/matches` | Работодатель | Подборка и банк кандидатов. |
| `/app/invitations` | Обе | Предложения и статусы. |
| `/app/vacancies` | Обе | Поиск вакансий или управление своими вакансиями. |
| `/app/applications` | Специалист | Собственные отклики. |
| `/app/tasks` | Обе | Задание недели либо задания компании и оценка ответов. |

### 3.3 Авторизованные запросы

Кабинет `/app` работает с `ssr=false`: загрузчик layout проверяет браузерную сессию, необходимость подтверждения email, выбор роли и доступность раздела. API cookie принадлежит origin API и передаётся браузером через `credentials: "include"`.

`shared/auth/index.ts` добавляет `X-CSRF-Token` к изменяющим запросам; JSON Content-Type ставится автоматически, кроме FormData. Ответ 401 сбрасывает локальное состояние авторизации. Ошибки сервера преобразуются в сообщения интерфейса.

`VITE_API_URL` встраивается в bundle при сборке. По умолчанию используется `http://localhost:8080`; пустая строка в production означает запросы к тому же origin. Изменение переменной после сборки не меняет уже выпущенный клиент.

### 3.4 UI и PDF-экспорт

Локальные Nunito и Play используются в интерфейсе. Иконки Moving Icons/Lucide и GSAP обеспечивают анимации, с учётом reduced motion. Экспорт резюме работает в браузере через pdfmake и Montserrat: API передаёт сохранённый профиль, серверный endpoint генерации готового PDF не используется.

## 4 Модель данных PostgreSQL

### 4.1 Группы таблиц и ключи

| Таблицы | Содержание и связи |
|---|---|
| `accounts` | UUID, уникальный subject, роль и дата создания. Корень прикладного аккаунта. |
| `auth_identities` | Провайдер, issuer, external_subject, email и флаг подтверждения; nullable FK на accounts. Уникальность тройки внешней идентичности. |
| `local_credentials` | Один credential на локальную идентичность: password_hash и состояние одноразовой верификации. |
| `browser_sessions`, `oauth_transactions` | Хэши session/state, сроки действия, CSRF; OAuth verifier и nonce, связь с браузерной сессией. |

| | |
|---|---|
| `candidate_profiles` | 1:1 к accounts; контакты, согласия, специализация, отрасль, desired_grade, JSONB опыта и навыков, privacy, привязка реестра ФСП. |
| `employer_companies`, `employer_needs` | 1:1 к аккаунту работодателя; компания, контакты и описание потребности. |
| `resume_drafts` | Распознанные поля JSONB, candidate_id, uploaded_at и confirmed_at. |
| `assessment_attempts` | Кандидат/специализация, опрос и blueprint JSONB, целевой грейд, сроки, theta/SE, score, passed/confident, сохранённый грейд и версия движка. |
| `assessment_items` | PK attempt_id + position; family_id, seed, view JSONB, серверный answer_key, допуск, ответ и время. |
| `questions`, `assessment_answers` | Данные прежнего пяти-вопросного движка; новый CAT не раздаёт этот банк. |
| `achievements` | Принадлежность кандидату, источник, внешний ID, событие, дисциплина, дата, уровень, место и баллы. |
| `fsp_registry_participants`, `fsp_registry_results` | Синтетические записи демонстрационного адаптера реестра. |
| `invitations` | FK работодателя и специалиста, предложение, вилка зарплаты, контакт компании, статус и даты. |
| `vacancies`, `vacancy_applications` | Вакансия работодателя и уникальный отклик пары vacancy_id/candidate_id. |
| `short_tasks`, `short_task_submissions`, `short_task_offers` | Задания, ответы с оценкой и стабильное назначение на период candidate_id/period_start. |

## 4.2 Логические связи

```
auth_identities N → 0..1 accounts
accounts 1 → 1 candidate_profiles ИЛИ employer_companies
candidate_profiles 1 → N assessment_attempts
assessment_attempts 1 → N assessment_items
employer_companies 1 → N invitations ← N candidate_profiles
employer_companies 1 → N vacancies
vacancies 1 → N vacancy_applications ← N candidate_profiles
employer_companies 1 → N short_tasks
short_tasks 1 → N short_task_submissions ← N candidate_profiles
```

Листинг 1. Основные связи прикладной модели. Nullable account_id позволяет выбрать роль после регистрации.

Грейд и балл текущего профиля выводятся из завершённых попыток текущей специализации, а не хранятся как редактируемые столбцы профиля. Выбирается наивысший подтверждённый грейд, затем лучший балл в этом грейде. Поле `desired_grade` независимо от подтверждения.

## 4.3 Ограничения и конкурентный доступ

- Частичный уникальный индекс допускает только одну незавершённую попытку на кандидата.
- `assessment_items` уникальны по позиции внутри попытки.

- Локальный email уникален без учёта регистра только для provider=local.
- Один FSP ID может принадлежать только одному профилю.
- Отклик уникален по вакансии и кандидату; ответ — по задаче и кандидату; недельный offer — по кандидату и началу недели.
- Зарплата положительна, верхняя граница не ниже нижней; rating находится в диапазоне 1–5.
- Для одноразовых OAuth/verification токенов и операций с попытками используются транзакции и блокировки строк.

FK-каскады очищают дочерние строки там, где это задано миграциями. Отзыв согласия дополнительно явно удаляет отклики/задания, приглашения, попытки, достижения и черновики, затем очищает профиль в одной транзакции. Сам прикладной аккаунт сохраняется.

### 4.4 Миграции

| Версия | Изменения |
|---|---|
| V1 | Аккаунты, профили, компании, потребности, старый банк вопросов, попытки, достижения и приглашения. |
| V2 | Обработка данных, CV/навыки JSONB, черновики резюме. |
| V5 | Вакансии, отклики, короткие задания, ответы и недельные offers. |
| V6 | Приватность, расширенные контакты компании, стек потребности, структурированные достижения и демо-реестр. |
| V7 | Адаптивные попытки и assessment_items; старые результаты сохраняются. |
| V8 | Browser auth: внешние/локальные identities, credentials, sessions и OAuth-транзакции. |
| V9 | Неудачная попытка не создаёт подтверждение Junior; исторические подтверждения пересчитываются. |
| V10 | Хэш, срок и время отправки email verification token. |

Версии перечислены так, как они присутствуют в репозитории; номера V3/V4 отсутствуют. Применённые migration-файлы не редактируются: новые изменения оформляются новой миграцией.

# 5 Идентификация, сессии и доступ

## 5.1 Локальная регистрация

Email обрезается и приводится к нижнему регистру. При регистрации разрешены 3–320 символов email и 12–128 символов пароля. Пароль хранится как PBKDF2-HMAC-SHA256: 600 000 итераций, случайная 16-байтовая соль, 256-битный результат. Формат хэша содержит алгоритм и стоимость. Ошибка неизвестного пользователя и неверного пароля одинакова; dummy hash поддерживает одинаковый путь проверки.

Регистрация создаёт идентичность и credential, ротирует сессию и отправляет письмо. Локальная неподтверждённая идентичность может войти для повторной отправки письма, но ресурсные маршруты и выбор роли требуют подтверждения почты.

## 5.2 Подтверждение email

Токен — 32 случайных байта в Base64URL без padding (43 символа). В БД хранится SHA-256-хэш; срок — 24 часа. Новая отправка заменяет предыдущий токен, интервал между отправками — минимум одна минута. Потребление выполняется один раз и очищает состояние верификации.

Ссылка подтверждения не является логином: в анонимном браузере она подтверждает почту, но не создаёт авторизованную сессию. В исходном авторизованном браузере сохраняется текущая идентичность и ротируется сессия. Login/register/verify/resend используют общий rate limit 5 запросов в минуту на IP процесса API.

Если SMTP отправка при регистрации не удалась, аккаунт уже создан: сервер отвечает 503 с пояснением, UI позволяет войти и повторить отправку. Парольный reset локального аккаунта в текущем интерфейсе/API не реализован.

## 5.3 Cookie и CSRF

| Параметр | Значение |
|---|---|
| Имя cookie | `career_session` |
| Атрибуты | HttpOnly, Path=/api, SameSite=Lax, host-only; Secure при APP_ENV=production. |
| Срок | 7 дней в cookie и PostgreSQL. |
| Хранение | Непрозрачный ID в браузере; SHA-256 ID в browser_sessions, отдельный csrf_token. |
| Изменяющие запросы | Cookie + Origin, точно совпадающий с FRONTEND_ORIGIN, + X-CSRF-Token. Для native формы привязки — поле csrfToken. |
| Ротация | При входе, выходе и подтверждении email старый session ID отзывается. |

CORS разрешает credentials только заданному frontend origin. Клиент не хранит пароль или токен провайдера в localStorage. Изменяющие bearer-only запросы без session cookie сохраняют bearer-путь; наличие cookie активирует CSRF-проверку.

## 5.4 ФСП ID / OIDC и VK ID

Сервер инициирует Authorization Code + PKCE S256, сохраняет state, verifier и nonce в PostgreSQL на 5 минут и связывает их с браузерной сессией. Callback одноразово потребляет state. Неправильная сессия или повторный callback не выполняют повторный вход.

Для ФСП ID проверяются подпись RS256 ID token, issuer, audience, exp, nonce и subject. Подтверждённый email принимается только при `email_verified=true`. Публичный issuer проверяется независимо от приватного URL JWKS.

Для VK ID сервер использует `service_token`, `device_id`, `code_verifier` и state; проверяет совпадение стабильного user_id в token response и user_info. Текущий VK-адаптер возвращает пустой email и verified=false. Контактную почту такой пользователь заполняет в профиле.

Одинаковая почта разных провайдеров не объединяет аккаунты. Явная привязка через POST `/api/auth/{provider}/link` требует авторизованной сессии, CSRF и повторного OAuth-подтверждения. Внешняя идентичность, уже принадлежащая чужому аккаунту, не переносится. Привязка входа и импорт достижений независимы.

## 5.5 Прикладные роли и владение

Роль хранится в accounts; инфраструктурный `account(subject, requiredRole)` проверяет её в транзакции. Репозитории вакансий, заданий и приглашений дополнительно ограничивают запросы владельцем/участником. Клиентские меню — удобство навигации; серверные проверки остаются определяющими.

Для legacy bearer-токенов Ktor проверяет подпись, issuer, audience и срок, получает ключи из кешируемого JWKS. Прежний аккаунт может быть сопоставлен только по проверенным issuer+subject, а не по пользовательскому email.

# 6 Профили, приватность и резюме

## 6.1 Профиль и согласия

PUT `/api/candidates/me` требует `processingConsent=true`. Контактный email валидируется по формату и длине до 320 символов; пробелы по краям убираются. Если поле пустое, используется подтверждённый email активной идентичности, если он есть. Непустой контакт может отличаться от логина.

Основные ограничения: имя до 200 символов, телефон до 50, специализация/отрасль до 100, опыт до 4000; списки технологий, ролей и soft skills — до 30 элементов по 100 символов; история работы — до 20 записей. Грейд принимает junior/middle/senior. Поля grade/score вычисляются из истории и не позволяют повысить категорию через JSON профиля.

Публикация допускается только по отдельному `consentPublication`. Отключение публикации убирает профиль из общего опубликованного пула. Для работодателя с уже раскрытыми контактами карточка может оставаться доступной по разрешённому сервером сценарию.

## 6.2 Приватность и контакты

JSONB privacy содержит fullName, techStack, experience, cvHistory, softSkills, fspAchievements. Проекция сначала применяет эти настройки, затем передаёт сигналы движку ранжирования. Скрытые данные не должны повышать рейтинг или попадать в объяснения.

Отдельная очистка текста `employerText` удаляет контакты и скрытое имя из свободных полей. Это позволяет не раскрывать email через описание опыта или ответ на короткое задание. Категория и результат оценки остаются основой выдачи.

## 6.3 Импорт PDF

POST `/api/candidates/me/resume-upload` принимает multipart `file`: PDF до 5 МБ, от 1 до 30 читаемых страниц. Проверяются PDF-сигнатура, размер, количество страниц, шифрование и наличие извлекаемого текста. PDFBox извлекает строки, имя, email/телефон и известные технологии; это эвристическое распознавание, OCR сканов отсутствует.

Файл не сохраняется: в resume_drafts записываются распознанные поля JSONB. Загрузка требует ранее сохранённого согласия и не обновляет профиль. POST `/api/candidates/me/resume-confirm` принимает draftId и исправленные поля; подтверждение возможно владельцем незавершённого черновика в течение суток. Контактный email, публикация и грейд этой операцией не заменяются.

## 6.4 Экспорт и отзыв согласия

pdfmake в браузере строит PDF из сохранённого профиля; шрифты загружаются локально. Это самопрезентация специалиста, а не официальное удостоверение грейда или достижений.

DELETE `/api/candidates/me/consent` удаляет относящиеся к кандидату отклики, offers и ответы на задания, приглашения, попытки с дочерними ответами, достижения и черновики, затем очищает профиль и согласия. Учётная запись/способ входа остаются; дальнейшая работа требует нового согласия. Это операция отзыва обработки, а не отдельный endpoint сброса теста.

# 7 Квалификационная оценка: IRT и CAT

## 7.1 Формат и жизненный цикл

Движок `cat-v1` обслуживает backend, frontend, data, product и design. Справочники отраслей, ролей и форматов выдаются каталогом. Опрос хранится вместе с попыткой; при старте его специализация/отрасль и targetGrade обновляют профиль.

```
каталог + опрос → eligibility → start attempt
→ текущее задание → ответ → следующее задание
→ результат → история + подтверждённая категория
```

Тест содержит 12 заданий, срок 30 минут. Задания выдаются последовательно; возврат и пропуск отсутствуют. Сервер принимает attemptId, текущую position и answer. Неверная позиция/повторный ответ отклоняются. Ключ, допуск и seed хранятся сервером и не входят в клиентскую модель.

Семейство содержит навык, тему и параметры сложности. CSPRNG seed воспроизводит конкретные числа, таблицы или код для аудита. В одной попытке семейство не повторяется; blueprint задаёт квоты тем.

## 7.2 Модель вероятности и оценка

Для параметров семейства a (различающая способность), b (сложность), c (нижняя асимптота) вероятность правильного ответа:

$$P(\theta) = c + \frac{1 - c}{1 + e^{-a(\theta - b)}}$$

Числовые задания используют c=0; выбор из вариантов — фиксированное c, соответствующее угадыванию. Логарифмическое правдоподобие аккумулируется по сетке theta от −4 до +4 с шагом 0,05 (161 точка). EAP — среднее апостериорного распределения; SE в реализации — его стандартное отклонение.

Для выбора следующего задания используется априорное N(anchor(target), 1). Для итогового результата — N(0, 1), чтобы заявленный уровень направлял последовательность, но не сдвигал оценку.

Следующее семейство выбирается среди ещё не использованных и с незаполненной квотой темы: вычисляется информация Фишера в текущей routing estimate, оставляются семейства с информацией не ниже 0,85 максимума, затем выбирается наименее часто показывавшееся; при равенстве — случайно.

Итоговый балл:

$$\text{score} = \text{round}\left(100 \cdot \Phi\left(\hat{\theta}\right)\right)$$

Здесь Phi — функция распределения стандартной нормальной шкалы. Балл 0–100 не равен проценту верных ответов. Верные ответы и число отвеченных задач отображаются отдельно.

## 7.3 Подтверждение грейда

| Грейд | Порог theta | Баллов на пороге | Routing anchor |
|---|---|---|---|
| Junior | −1,5 | 7 | −1,0 |
| Middle | −0,5 | 31 | 0,15 |
| Senior | 0,8 | 79 | 1,4 |

Условие успеха: theta_hat − K·SE ≥ cut(target), K=0. Уверенный результат дополнительно достигает порога следующего уровня, K_UPGRADE=0. Это открывает следующий тест, а не автоматически присваивает следующий грейд.

После успеха сохраняется максимум предыдущего и подтверждённого целевого грейда. Неудача не понижает существующее подтверждение и не создаёт Junior для пользователя без успешной истории. При истечении времени unanswered задания учитываются

как неудачные на заявленном уровне; expired=true принудительно выключает passed/confident.

Численные параметры — настройки реализованного прототипа. Наличие алгоритма и синтетических симуляций не заменяет экспертную калибровку на реальной популяции.

### 7.4 Пересдачи

Правило применяется отдельно по специализации. Основа — последняя завершённая попытка; ожидание составляет 90 дней от её submittedAt.

| Состояние | Доступность |
|---|---|
| Истории нет | Можно выбрать любой уровень. |
| Прошло 90 дней | Любой уровень не ниже уже подтверждённого. |
| Последняя попытка неудачна | Сразу доступен уровень ровно на одну ступень ниже, если он не ниже подтверждённого. У Junior нижней ступени нет. |
| Последний результат уверенный | Сразу доступен уровень ровно на ступень выше, если этот уровень не был провален за последние 90 дней. |
| Успех без уверенного результата | Новая оценка ожидает окончания 90 дней. |
| Незавершённая попытка | Новая не создаётся; текущую продолжают либо она закрывается по сроку. |

Eligibility возвращает allowed, reason, message, availableAt для каждого уровня. Start повторяет проверку сервером: визуально активная кнопка не отменяет ограничение. После follow-up он становится последней попыткой; цепочка не позволяет зациклить Senior → Middle → Senior после недавнего провала.

### 7.5 Blueprint по вакансии

POST `/api/assessments/blueprint` разбирает готовую demo-вакансию или vacancyText, выделяет навыки по словарю, строит квоты и выдаёт примеры. Для этого preview 60% веса тем берётся из вакансии, 40% — базовый профиль направления. Квалификационный старт не принимает vacancyId/vacancyText: он строится только под категорию.

Данные legacy-v1 сохраняются для истории; новый интерфейс не отправляет старые пяти-вопросные тесты. При обнаружении открытой попытки старого движка она удаляется, поскольку прежний submit endpoint больше не обслуживается.

Источник: `assessment/domain/{Irt,Cat,RetakePolicy,AssessmentPlanning}.kt`, `assessment/infrastructure/AssessmentPersistence.kt`.

# 8 Объяснимый подбор кандидатов

### 8.1 Подготовка сигналов и фильтры

SQL-проекция формирует опубликованный пул с processingConsent и подтверждённой категорией. Приватность применяется до ранжирования. Категория строится по успешным результатам в текущей специализации.

В режиме потребности специализация должна совпасть. Коэффициент соответствия грейда g: точное совпадение 1,00; выше на одну ступень 0,85; выше на две 0,70. Ниже исключается, кроме одной ступени при allowLowerGrade=true, тогда 0,60.

Общие фильтры: специализация/категория, грейд, стек (все перечисленные навыки), достижения, отрасль и текст. Alias-normalization делает эквивалентными известные варианты имён технологий. Скрытый стек не участвует в фильтре.

## 8.2 Рейтинг по потребности

$$R = g \cdot \sum_i w_i x_i, \quad x_i \in [0, 1]$$

| Фактор | Вес | Расчёт |
|---|---|---|
| Тест | 30 | Подтверждённый балл / 100. |
| Текст | 25 | TF-IDF cosine между потребностью и narrative профиля / 0,35, ограниченный 1. |
| Стек | 20 | Среднее покрытие требуемых навыков с учётом подтверждения описанием. |
| ФСП | 15 | Сумма баллов видимых подтверждённых достижений с recency / 20, максимум 1. |
| Отрасль | 5 | 1 при совпадении; 0,5 при упоминании в тексте. |
| Актуальность | 5 | 1 при активности за 30 дней; линейно до 0,3 за год. |

Когда потребность не содержит текста, стека или отрасли, неприменимые факторы убираются, а оставшиеся веса нормируются до 100. Объяснение содержит code/label, weight, value, points и текст. Итог умножается на g. Порядок при равенстве: relevance, strength, стабильный candidate ID.

TF-IDF-корпус готовится один раз на запрос по опубликованному пулу. Для рейтинга narrative содержит опыт, CV, роли и soft skills; теги стека исключены из narrative, чтобы не начислять за один тег два фактора. Для текстового фильтра банка используется отдельное представление narrative + теги.

## 8.3 Покрытие стека и согласованность

За требуемый навык начисляется: 1,0 при доступном подтверждении оценкой; 0,8 при перечислении и описании в narrative; 0,5 при одном описании; максимум 0,5·r за голый тег. Коэффициент:

$$r = \frac{\text{corroborated} + 1}{\text{listed} + 2}$$

Это сглаженная доля тегов, подтверждённых narrative. Длинный список технологий без соответствующего опыта получает меньше веса. Порт confirmedSkills предусмотрен моделью сигналов; если интеграция не передала такие данные, применяется имеющееся самоописание.

## 8.4 Банк кандидатов и пагинация

$$S = 65 \cdot x_{\text{test}} + 25 \cdot x_{\text{fsp}} + 10 \cdot x_{\text{freshness}}$$

В банке текстовый запрос фильтрует, но не заменяет порядок по силе профиля. Сортировка: strength, балл теста, candidate ID. Категории содержат количество специалистов и средние показатели. Клиент хранит полученные выборки банка и потребности отдельно; локальное уточнение не теряет исходный набор.

Ответ MatchResponse содержит matches, categories, formula, total и nextOffset. `limit` по умолчанию 100, максимум 500; `offset` задаёт следующую порцию. Локальные поиск/сортировка применяются только к загруженным строкам, серверные параметры — ко всему отобранному пулу.

ФСП-результаты учитываются полностью первые два года, затем вес уменьшается на 20% в год с нижней границей 40%. Отсутствие достижений означает нулевой вклад фактора и не меняет подтверждённый грейд.

Источник: `matching/domain/MatchingEngine.kt`, `matching/infrastructure/MatchingProjection.kt`.

# 9 Приглашения, вакансии и регулярные задания

## 9.1 Приглашения

Работодатель указывает candidateId, описание, salaryMinRub, salaryMaxRub и contactMethod. Вилка обязательна, в рублях за месяц до НДФЛ. Метод связи по умолчанию собирается клиентом из компании и сохраняется в предложении.

```
sent → viewed → accepted
sent ──────────→ accepted
sent / viewed → rejected
```

Состояние меняет приглашённый кандидат. До accepted ответы для работодателя не содержат контакт специалиста; после accepted контакты открываются только этой компании. Статусы и зарплаты видны обеим сторонам; метод связи работодателя виден кандидату ещё до решения.

## 9.2 Вакансии и отклики

Вакансия содержит специализацию, требуемый грейд, стек, зарплату, формат office/remote/hybrid, город и описание. Для создания требуется непустое название компании. Статусы open/closed переключаются явными close/reopen; закрытие сохраняет отклики.

Поиск кандидата учитывает текст, стек, формат и salaryFrom. Этот фильтр сравнивается с верхней границей вилки. Рассматриваются до 500 новых открытых вакансий перед skill/text-фильтрами. Порядок учитывает категорию, специализацию, пересечение стека и свежесть.

Отклик требует согласия, имени, специализации, минимум одного контакта и подтверждённого грейда текущего направления. Отправка раскрывает компании контактные поля в проекции отклика независимо от сценария приглашения.

```
sent → viewed → accepted или rejected
sent / viewed → DELETE (отзыв специалистом)
```

accepted/rejected являются конечными решениями. UI работодателя при открытии новых откликов отправляет переход viewed; простой серверный GET списка сам по себе не описывается как команда просмотра. Отзыв удаляет ожидающий отклик и доступ через него; уже принятые приглашения остаются отдельным основанием доступа.

## 9.3 Еженедельное назначение

Неделя начинается в понедельник 00:00 UTC. При первом запросе current task сервер выбирает активную задачу своей специализации и подходящего грейда: подтверждённого, либо desired до оценки. grade=null подходит всем. Уже отвеченные задания исключаются.

Выбор детерминирован по SHA-256 кандидата и недели над отсортированным пулом, затем сохраняется в short_task_offers. Добавление задач в середине недели не заменяет текущую. Перевыбор происходит при снятии задачи или изменении специализации. После ответа слот остаётся answered до следующей недели.

Ответ имеет kind=solution/approach и до 8000 символов. Автор компании видит имя, категорию и очищенный текст без контактов. Оценка 1–5 и комментарий до 1000 символов сохраняются отдельно; повторная оценка заменяет прошлую. Активность отражается в профиле/подборе, а rating не становится квалификационным грейдом.

Источник: `recruitment/domain/RecruitmentPolicy.kt`, `recruitment/infrastructure/{TaskPersistence,VacancyPersistence,InvitationPersistence}.kt`.

# 10 Реестр достижений и интеграционные порты

## 10.1 Текущий адаптер ФСП

`FspRegistry` — порт поиска участника и его результатов. `MockFspRegistry` читает синтетические таблицы V6; issuerLabel — «ФСП · демо-реестр». Это реализованный демонстрационный источник, отдельный от OAuth входа через ФСП ID.

Импорт требует processingConsent и подтверждённый email активной идентичности, совпадающий с email участника без учёта регистра. Произвольный contact email из профиля не используется как доказательство владения. Неизвестная и чужая запись дают одинаковый 404. Повторный импорт обновляет ранее импортированные результаты; отвязка удаляет source=fsp, сохраняя способ входа.

Базовые баллы: региональный уровень 4, федеральный 7, международный 10. Множитель результата: победитель 1,6; призёр 1,3; финалист 1; участник 0,5. После округления значение ограничено 1–20; recency применяется при подборе, а не сохраняется в points.

Для подключения настоящего реестра заменяется адаптер порта и схема доказательства принадлежности; реальные URL/claims/credentials определяются контрактом провайдера. Записи демонстрационного реестра не обозначают официальные достижения реальных пользователей.

## 10.2 SMTP

VerificationMailer отделяет use case от транспорта. SMTP-адаптер использует Jakarta Mail; конфигурация задаёт host, port, from, credentials и STARTTLS. В homelab API и Keycloak доставляют сообщения в Mailpit внутри namespace. Mailpit не пересылает их во внешние почтовые ящики; тестовый inbox доступен по `/mail/`, хранилище эфемерно.

## 10.3 PDF и внешняя идентичность

ResumeDocuments инкапсулирует извлечение PDF; PasswordHasher — вычисление/проверку хэша; IdentityRepository — сессии, токены и привязки. Подмена этих интерфейсов в тестах позволяет выполнять use cases без настоящего SMTP/OAuth-провайдера. HTTP integration tests используют MockEngine и тестовый PostgreSQL.

# 11 HTTP API и примеры контрактов

## 11.1 Общие правила

Публичные точки — `/health`, `/openapi.yaml` и browser-auth endpoints со своей CSRF-политикой. Ресурсные `/api/*` используют cookie либо проверенный bearer JWT. UUID приходит в path или JSON; выбор текущего владельца производится по сессии, а не произвольному accountId клиента.

Ошибка обычно возвращается как JSON `{"error":"..."}`. Значения 400 — неправильный ввод/формат; 401 — нет действительной авторизации; 403 — роль/согласие/CSRF/верификация; 404 — нет доступного ресурса; 409 — недопустимое состояние; 429 — rate limit; 503 — сбой отправки письма. OAuth-ошибки перенаправляют на UI с кодом состояния. Необработанное исключение логируется сервером и отдаёт обезличенный 500.

`/health` возвращает `{"status":"ok"}` и является liveness процесса; он не выполняет отдельный запрос доступности БД или внешнего OAuth. Полный каталог операций и задекларированных ответов приведён в приложении.

## 11.2 Получение сессии и изменение профиля

Сначала GET `/api/auth/session` возвращает cookie и csrfToken; после входа токен обновляется. Для изменяющего запроса браузера:

```
PUT /api/candidates/me HTTP/1.1
Origin: https://brokenbrain.online
Content-Type: application/json
Cookie: career_session=<browser-cookie>
X-CSRF-Token: <token-from-current-session>

{
  "fullName": "Демо Специалист",
  "email": "contact@example.org",
  "specialization": "backend",
  "industry": "fintech",
  "desiredGrade": "junior",
  "techStack": ["Kotlin", "PostgreSQL"],
  "processingConsent": true,
  "consentPublication": true
}
```

Пример намеренно содержит контактный email; логин и verified-флаг идентичности не обновляются.

## 11.3 Старт и ответ в оценке

```
{
  "survey": {
    "specialization": "backend",
    "industry": "fintech",
    "experienceYears": 2,
    "teamRoles": [],
    "workFormat": "remote"
  },
  "targetGrade": "junior"
}
```

Тело отправляется в POST `/api/assessments/attempts`. На GET eligibility specialization передаётся query-параметром. POST answers получает текущую позицию и строковый ответ:

```
{"position": 0, "answer": "a"}
```

Для числового задания answer тоже строка, например `"42"`. Ответ сервера содержит либо следующее attempt/item, либо done=true и result. Ключ и промежуточная правильность не возвращаются.

## 11.4 Поиск и предложение

```
GET /api/matches?allowLowerGrade=true&limit=20&offset=0
GET /api/bank/candidates?specialization=backend&grade=junior
```

```
{
  "candidateId": "<uuid-from-selection>",
  "description": "Backend-разработчик платёжного сервиса",
  "salaryMinRub": 120000,
  "salaryMaxRub": 180000,
  "contactMethod": "hr@example.org"
}
```

Второе тело отправляется в POST `/api/invitations`. ID из ответов используются в view/accept/reject. Для vacancy apply отправляется coverNote; изменение статуса отклика работодателем — `{"status":"accepted"}`.

### 11.5 Основные типы

| Модель OpenAPI | Назначение |
|---|---|
| `BrowserSession`, `Me` | Состояние входа, CSRF, провайдеры, verification-required и прикладная роль. |
| `Candidate`, `CandidatePrivacy` | Профиль и видимость разделов. |
| `ResumeDraft`, `ConfirmResume` | Черновик и проверенные поля импорта. |
| `AssessmentAttempt`, `AssessmentResult` | Текущее задание и итоговая квалификационная оценка. |
| `MatchingResponse`, `MatchingCandidateCard` | Рейтинг, категории, объяснения и фильтрованная карточка. |
| `Invitation`, `VacancyRequest` | Предложение и входная модель вакансии. |
| `CurrentShortTask`, `TaskSubmissionRequest` | Назначение недели и ответ специалиста. |

Полные nullable/readOnly поля, JSON Schema, query-параметры и ограничения следует брать из OpenAPI рядом с версией приложения.

# 12 Конфигурация и локальный запуск

### 12.1 Переменные окружения

| Переменная | Назначение |
|---|---|
| `DATABASE_URL`, `DATABASE_USER`, `DATABASE_PASSWORD` | JDBC URL и подключение PostgreSQL приложения. |
| `PORT` | Порт API, по умолчанию 8080. |
| `KEYCLOAK_ISSUER`, `KEYCLOAK_AUDIENCE` | Обязательный issuer и audience legacy bearer-верификации. |
| `KEYCLOAK_JWKS_URL` | Приватный/явный JWKS URL; публичный issuer при этом сохраняется. |
| `FRONTEND_ORIGIN`, `API_ORIGIN` | Точные origins CORS/CSRF и OAuth callbacks. Production требует явные HTTPS origins. |
| `APP_ENV` | `production` включает Secure cookie и проверки HTTPS. |
| `FSP_CLIENT_ID`, `FSP_CLIENT_SECRET` | Включают серверный OIDC-вход при наличии обоих. |
| `FSP_ISSUER`, `FSP_AUTH_URL`, `FSP_TOKEN_URL`, `FSP_JWKS_URL` | Публичный issuer/auth и back-channel URL провайдера. |
| `VK_CLIENT_ID`, `VK_CLIENT_SECRET` | VK application ID и service token; оба нужны для включения. |
| `SMTP_HOST`, `SMTP_PORT`, `SMTP_FROM` | Адрес и отправитель verification mail. |

| | |
|---|---|
| `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_STARTTLS` | Опциональная авторизация и TLS SMTP. |
| `VITE_API_URL` | Build-time frontend API origin; пустая строка — same-origin. |
| `ORIGIN`, `HOST`, `PORT` web | Runtime origin и bind adapter-node. По умолчанию контейнер слушает 3000. |
| `TEST_DATABASE_*` | Изолированная БД интеграционных тестов. |

OAuth callbacks: `/api/auth/fsp/callback` и `/api/auth/vk/callback` относительно API_ORIGIN. FSP-приложению нужны Authorization Code и PKCE S256, confidential client, scopes openid/email, подписанный ID token. Origin не допускает user-info/query/fragment или произвольного пути.

## 12.2 Полный локальный стек

Нужны Docker Engine и Compose v2. `.env.example` — шаблон параметров; `.env` не коммитится.

```
cp .env.example .env
# Установить свои локальные пароли и нужные origins в .env.
docker compose up --build -d
docker compose ps
curl --fail http://localhost:8080/health
```

| Сервис | Локальный адрес |
|---|---|
| Web | `http://localhost:3000` |
| API / OpenAPI | `http://localhost:8080` · `/openapi.yaml` |
| Keycloak | `http://localhost:8081` |
| Mailpit UI | `http://localhost:8025` |
| PostgreSQL | `localhost:5432` |

Keycloak realm импортируется только в пустую БД. После запуска провайдер ФСП ID включается настройкой confidential fsp-service и соответствующего secret. Локальные письма API/Keycloak приходят в Mailpit. SMTP-порт Mailpit внутри Compose — 1025; наружу опубликован UI, но не SMTP.

Volumes сохраняются при обычном `docker compose down`. Команда с `-v` удаляет БД. Примерные origins и redirect URI в realm относятся к указанным портам.

## 12.3 Разработка вне контейнеров

Нужны JDK 21, Bun 1.4.2 и Gradle (или checked-in wrapper). Сначала запускаются PostgreSQL/Keycloak/Mailpit, затем экспортируются переменные для host API. Compose не экспортирует `.env` в текущую оболочку.

```
docker compose up -d postgres keycloak-db mailpit keycloak
```

В рабочем каталоге `apps/api`:

```
export DATABASE_URL=jdbc:postgresql://localhost:5432/fsp_career
export DATABASE_USER=fsp_app
export DATABASE_PASSWORD='<значение из локального .env>'
export KEYCLOAK_ISSUER=http://localhost:8081/realms/fsp-career
export KEYCLOAK_AUDIENCE=fsp-api
```

```
export FRONTEND_ORIGIN=http://localhost:5173
export API_ORIGIN=http://localhost:8080
./gradlew run
```

Для отправки verification mail host-процессу дополнительно нужен доступный SMTP endpoint: Compose-only имя `mailpit` не является host DNS, а 1025 по умолчанию не опубликован. Подключение SMTP настраивается отдельно.

В рабочем каталоге `apps/web`:

```
bun install --frozen-lockfile
bun run dev
```

API image/CI используют Gradle 9.1/JDK 21; checked-in wrapper указывает 9.6.0. Обновление wrapper и CI следует согласовывать отдельно от бизнес-изменений.

# 13 Развёртывание и CI/CD

## 13.1 Активная топология

Рабочий origin — `https://brokenbrain.online`. Web, API, Keycloak и Mailpit запускаются в K3s namespace fsp-career; Envoy Gateway завершает HTTPS с сертификатом cert-manager. PostgreSQL приложения и Keycloak размещены отдельно на goodman и используют разные БД/роли.

| Путь | Назначение |
|---|---|
| `/` | SvelteKit web и статические assets. |
| `/api/*`, `/health`, `/openapi.yaml` | Ktor API. |
| `/auth/*` | Keycloak с темой ФСП. |
| `/mail/` | Демонстрационный Mailpit inbox. |

Kustomize-наборы data → identity → identity-config → app применяются Flux последовательно. Runtime-компонент задаёт общие ограничения. Платформенный репозиторий владеет namespace, RBAC, admission, квотами, NetworkPolicy и registry access; этот проект — только разрешёнными namespace-scoped workload-ресурсами.

Контейнеры работают под непривилегированным runtime account без смонтированного Kubernetes API token. Секреты окружения хранятся SOPS/age-шифрованными в data/secrets; открытые параметры SMTP определены API Deployment. Пароли и OAuth secret не включаются в frontend VITE-переменные или документацию.

## 13.2 Pipeline GitHub Actions

```
push main / pull_request / workflow_dispatch
→ web + api/PostgreSQL + gitops checks
→ publish API + web + Keycloak в GHCR (только main)
→ атомарный commit digests в deploy/homelab
→ Flux reconcile → Kubernetes rollout
```

Web job: Bun 1.4.2, frozen lockfile, Svelte check, Bun tests, ESLint, production build с VITE_API_URL=«». API job: JDK 21, Gradle 9.1.0, PostgreSQL 18 service, unit/architecture/database tests; отчёты сохраняются artifact. GitOps job проверяет promotion script и rendered Kustomize manifests.

Образы Linux amd64 имеют тег sha-коммита и registry digest. Promotion записывает digests всех трёх образов вместе в app/identity kustomization.yaml; при новых source-изменениях устаревший release не продвигается. Релизы сериализованы, cancel-in-progress=false, push-конфликты повторяются до пяти раз.

Push, меняющий только deploy/homelab, не запускает новую image-сборку. Изменение README или документации тоже запускает общий workflow. CI не содержит cluster credentials: успех promotion подтверждает запись release в Git, а применение проверяется Flux и состоянием workloads.

### 13.3 Релиз и откат

Для нового релиза коммитят source в main и проверяют все jobs, затем commit promotion, Flux revision и здоровье web/API. Для отката возвращают прежние image digests отменой соответствующего deployment promotion; схема БД при этом не откатывается автоматически.

Прежний standalone-вариант Compose/Traefik сохранён в `compose.prod.yaml` и `infra/deploy/deploy.sh`; активный pipeline доставляет приложение через GHCR/Flux.

## 14 Тестирование и техническая валидация

### 14.1 Уровни проверок

| Уровень | Покрытие |
|---|---|
| Domain/unit | Валидация, роли и переходы, IRT/EAP, CAT/blueprints, пересдачи, текстовый анализ, рейтинг и приватность. |
| Architecture | Запрет Ktor/SQL/PDFBox в domain/application, запрет внешних адаптеров в use cases. |
| HTTP + PostgreSQL | Регистрация/письма/сессии/CSRF, OAuth callbacks и link, миграции/DAO, оценка, подбор, вакансии и задания. |
| Web | Svelte/TypeScript check, Bun regression tests, ESLint, production build. |
| Deployment | Unit tests promotion и валидация namespace-scoped rendered manifests. |

### 14.2 Команды

В `apps/web`:

```
bun install --frozen-lockfile
bun run check
bun test
bun run lint
VITE_API_URL= bun run build
```

В `apps/api` с изолированной тестовой PostgreSQL:

```
export TEST_DATABASE_URL=jdbc:postgresql://localhost:5432/career_test
export TEST_DATABASE_USER=career
export TEST_DATABASE_PASSWORD='<пароль тестовой роли>'
./gradlew --no-daemon --console=plain test
```

Без TEST_DATABASE_URL интеграционные классы используют assumption и пропускаются. Успешный unit-run без БД не подтверждает PostgreSQL/HTTP-сценарии. Публичные OAuth-провайдеры в тестах заменяются MockEngine; реальный provider login проверяется отдельно с действующей конфигурацией.

GitOps-проверки из корня (Python 3/PyYAML и kubectl):

```
python3 -m unittest discover -s infra/deploy/tests -v
kubectl kustomize deploy/homelab/prod/app \
  | python3 infra/deploy/validate_manifests.py
```

Синтетические исследования алгоритмов доступны задачами `validateMatching` и `validateAssessment`. Они генерируют отчёты из симуляций, а не оценку реальных людей. Этот документ описывает воспроизводимые команды и проверки, без заявления о пройденной официальной калибровке или независимом security audit.

### 14.3 Регрессионные сценарии

1. Регистрация → одноразовое подтверждение → выбор роли → профиль с согласиями.
2. Сохранение другого contact email, чтение после reload; логин и emailVerified не меняются.
3. PDF upload → исправление draft → confirm; публикация/email не заменяются.
4. Start → последовательные ответы → результат; неудача не создаёт новый грейд и не понижает подтверждённый.
5. Повторный старт учитывает cooldown и исключения; истёкшая попытка закрывается.
6. Скрытые поля отсутствуют в карточке и факторах. До принятия invitation контакты отсутствуют, после — доступны нужной компании.
7. Самоотклик → viewed → accepted/rejected; pending withdraw убирает отклик.
8. Weekly offer стабилен; один ответ на период, rating доступен только автору задания.

## 15 Эксплуатация и диагностика

### 15.1 Проверки окружения

Команды выполняются с настроенным доступом к нужному кластеру:

```
kubectl -n flux-system get kustomization \
  fsp-career-data fsp-career-identity \
  fsp-career-identity-config fsp-career
kubectl -n fsp-career get pods
kubectl -n fsp-career rollout status deployment/fsp-career-api
kubectl -n fsp-career rollout status deployment/fsp-career-web
kubectl -n fsp-career logs deployment/fsp-career-api
curl --fail https://brokenbrain.online/health
```

Сопоставляют source/promotion SHA, image digest в manifests, observed revision Flux и фактическое состояние pod. Image publish и promotion сами по себе не являются подтверждением rollout. Логи API содержат детали необработанных исключений; браузер получает общий 500.

### 15.2 Типовые причины

| Симптом | Проверка |
|---|---|
| Письмо не пришло во внешний ящик | В текущем homelab используется Mailpit. Проверить `/mail/`, SMTP_HOST=fsp-career-mailpit, порт 1025 и состояние сервиса. |
| Регистрация 503 | Аккаунт может уже существовать. Войти, запросить verification-email; учитывать минутный интервал. |
| Провайдер «Не подключён» | Оба CLIENT_ID/CLIENT_SECRET в runtime API; session.providers отражает конфигурацию. |
| VK/OIDC не завершает вход | Точный callback URI, public issuer/audience, включение provider; новая OAuth-транзакция после провала. |

| | |
|---|---|
| 403 при сохранении | Session cookie, Origin, свежий CSRF, роль, processingConsent и подтверждение локального email. |
| Все грейды недоступны | Eligibility для текущей специализации, lastAttemptAt, reason и availableAt; 90 дней и confident-флаг. |
| Кандидата нет в подборке | Публикация/обработка, успешная категория текущей специализации, grade gate и фильтры. |
| Отклик недоступен | Имя, специализация, контакт и подтверждённый грейд; вакансия открыта, нет прежнего отклика. |
| Импорт ФСП не проходит | Email активной подтверждённой идентичности и email записи реестра; contact email не является подтверждением. |
| PDF не читается | Текстовый PDF, размер/страницы/шифрование; OCR не реализован. |
| Новое задание не появилось | Недельный offer стабилен и после ответа ждёт следующего понедельника UTC; проверить активность и специализацию. |

### 15.3 Сопровождение данных

До операций восстановления проверяется совместимость migration-версии и приложения. Backup/restore PostgreSQL относится к базе приложения и отдельно к базе Keycloak; удаление аккаунта провайдера не эквивалентно очистке прикладного профиля. Для повторяемой демонстрации новой попытки используется новый demo-аккаунт либо отдельная административная операция сопровождения: публичного endpoint сброса результатов нет.

Очистка персональных данных реализована штатным отзывом согласия. Ручные SQL-изменения должны учитывать FK, агрегаты попыток и основания раскрытия контактов; документ не предполагает общий TRUNCATE рабочего окружения.

## 16 Развитие и поддержка документации

### 16.1 Точки изменения

- Новая бизнес-политика добавляется в domain и используется application через существующие порты.
- Изменение хранения оформляется новой Flyway migration и DAO/projection mappings.
- Новый HTTP contract согласуется с OpenAPI, клиентскими types/API-обёртками и HTTP tests.
- При изменении CAT/границ обновляются Irt/RetakePolicy и регрессии, а не только текст интерфейса.
- Для настоящего реестра ФСП заменяется FspRegistry adapter и ownership-контракт.
- Для внешней почты меняются SMTP/runtime secrets и разрешённый egress; Mailpit остаётся тестовым transport.

### 16.2 Источники и сборка PDF

Исходник — `docs/documentation/project.typ`, сборка — `docs/documentation/build.sh`. Typst читает package metadata web и OpenAPI напрямую. Дата и source revision передаются параметрами. Статические шрифты Play/Montserrat берутся из проекта; code/math-шрифты входят в Typst. Системные и variable-шрифты исключены, сетевых пакетов Typst нет.

```
bash docs/documentation/build.sh
```

Выход — `docs/FSP-Career-documentation.pdf`. Для Nix: `nix shell nixpkgs#typst --command bash docs/documentation/build.sh`. Требуемая версия Typst — 0.14.2 или новее. Проверка PDF: pdfinfo, pdffonts, pdftotext и визуальный просмотр таблиц/схем.

Основные технические источники: `Application.kt`, context Dependencies, domain policies, infrastructure repositories, SQL migrations, `src/routes`, `shared/auth`, Compose, Dockerfile и `.github/workflows/deploy.yml`. Окружение уточняется по `docs/homelab.md` и `deploy/homelab/README.md`.

## 16.3 Ссылки

- Репозиторий проекта
- Развёрнутый прототип
- OpenAPI рабочего API
- CI/CD и отчёты проверок
- Скринкасты: Яндекс Диск
- Скринкасты: Google Диск

# 17 Приложение: каталог HTTP API

Каталог построен при сборке из OpenAPI 2.0.0. Количество операций: 62. Коды в таблице — ответы, задекларированные спецификацией; общие middleware также могут возвращать CSRF/rate-limit ошибки. Подробные payload и query schemas находятся в исходном OpenAPI. `{provider}` принимает fsp или vk.

| Метод | Путь | Коды ответов |
|---|---|---|
| **GET** | /api/auth/session | 200 |
| **POST** | /api/auth/register | 201, 400, 403, 429, 503 |
| **POST** | /api/auth/verify-email | 200, 400, 403, 429 |
| **POST** | /api/auth/verification-email | 204, 401, 403, 409, 429, 503 |
| **POST** | /api/auth/login | 200, 401, 403, 429 |
| **POST** | /api/auth/logout | 200, 403 |
| **GET** | /api/auth/{provider}/start | 302, 404 |
| **POST** | /api/auth/{provider}/link | 302, 401, 403, 404 |
| **GET** | /api/auth/{provider}/callback | 302 |
| **GET** | /health | 200, 500 |
| **GET** | /openapi.yaml | 200, 500 |
| **GET** | /api/me | 200, 401, 500 |
| **POST** | /api/me/role | 200, 400, 401, 409, 500 |
| **GET** | /api/candidates/me | 200, 401, 403, 500 |
| **PUT** | /api/candidates/me | 200, 400, 401, 403, 500 |
| **DELETE** | /api/candidates/me/consent | 200, 401, 403, 500 |
| **POST** | /api/candidates/me/resume-upload | 201, 400, 401, 403, 500 |
| **POST** | /api/candidates/me/resume-confirm | 200, 400, 401, 403, 404, 500 |
| **GET** | /api/assessments/catalog | 200, 401, 500 |
| **POST** | /api/assessments/blueprint | 200, 400, 401, 404, 500 |
| **GET** | /api/assessments/eligibility | 200, 400, 401, 403, 500 |
| **GET** | /api/assessments/history | 200, 401, 403, 500 |
| **POST** | /api/assessments/attempts | 201, 400, 401, 403, 404, 409, 500 |
| **GET** | /api/assessments/attempts/current | 200, 401, 403, 500 |
| **POST** | /api/assessments/attempts/{id}/answers | 200, 400, 401, 403, 404, 409, 500 |
| **GET** | /api/employers/company | 200, 401, 403, 500 |
| **PUT** | /api/employers/company | 200, 400, 401, 403, 500 |
| **GET** | /api/employers/needs | 200, 401, 403, 500 |
| **PUT** | /api/employers/needs | 200, 400, 401, 403, 500 |
| **GET** | /api/matches | 200, 400, 401, 403, 500 |
| **GET** | /api/bank/candidates | 200, 400, 401, 403, 500 |
| **GET** | /api/bank/candidates/{id} | 200, 400, 401, 403, 404, 500 |
| **GET** | /api/candidates/me/privacy | 200, 401, 403, 500 |
| **PUT** | /api/candidates/me/privacy | 200, 400, 401, 403, 500 |
| **GET** | /api/candidates/me/fsp | 200, 401, 403, 500 |

| Метод | Путь | Коды ответов |
|---|---|---|
| **POST** | /api/candidates/me/fsp | 200, 400, 401, 403, 404, 409, 500 |
| **DELETE** | /api/candidates/me/fsp | 200, 401, 403, 500 |
| **GET** | /api/invitations | 200, 401, 403, 500 |
| **POST** | /api/invitations | 201, 400, 401, 403, 404, 500 |
| **POST** | /api/invitations/{id}/view | 200, 400, 401, 403, 404, 409, 500 |
| **POST** | /api/invitations/{id}/accept | 200, 400, 401, 403, 404, 409, 500 |
| **POST** | /api/invitations/{id}/reject | 200, 400, 401, 403, 404, 409, 500 |
| **GET** | /api/employers/vacancies | 200, 401, 403, 500 |
| **POST** | /api/employers/vacancies | 201, 400, 401, 403, 500 |
| **PUT** | /api/employers/vacancies/{id} | 200, 400, 401, 403, 404, 500 |
| **POST** | /api/employers/vacancies/{id}/close | 200, 400, 401, 403, 404, 409, 500 |
| **POST** | /api/employers/vacancies/{id}/reopen | 200, 400, 401, 403, 404, 409, 500 |
| **GET** | /api/employers/vacancies/{id}/applications | 200, 400, 401, 403, 404, 500 |
| **PUT** | /api/employers/applications/{id}/status | 200, 400, 401, 403, 404, 409, 500 |
| **GET** | /api/vacancies | 200, 400, 401, 403, 500 |
| **POST** | /api/vacancies/{id}/applications | 201, 400, 401, 403, 404, 409, 500 |
| **GET** | /api/candidates/me/applications | 200, 401, 403, 500 |
| **DELETE** | /api/candidates/me/applications/{id} | 204, 400, 401, 403, 404, 409, 500 |
| **GET** | /api/employers/tasks | 200, 401, 403, 500 |
| **POST** | /api/employers/tasks | 201, 400, 401, 403, 500 |
| **POST** | /api/employers/tasks/{id}/deactivate | 200, 400, 401, 403, 404, 409, 500 |
| **POST** | /api/employers/tasks/{id}/activate | 200, 400, 401, 403, 404, 409, 500 |
| **GET** | /api/employers/tasks/{id}/submissions | 200, 400, 401, 403, 404, 500 |
| **PUT** | /api/employers/task-submissions/{id}/rating | 200, 400, 401, 403, 404, 500 |
| **GET** | /api/candidates/me/tasks/current | 200, 400, 401, 403, 500 |
| **GET** | /api/candidates/me/task-submissions | 200, 401, 403, 500 |
| **POST** | /api/candidates/me/task-submissions | 201, 400, 401, 403, 409, 500 |

Сгенерировано из `apps/api/src/main/resources/openapi.yaml`. Техническое описание поведения в основных разделах сверено с реализацией, включая адаптивный движок и редактируемую контактную почту.